DATA ENGINEERING & CLOUD · LESSON 9 OF 15

Streaming with Apache Kafka: Topics, Partitions, Consumers and Delivery Guarantees

Understand event streaming with Apache Kafka. Learn topics, partitions, offsets, consumer groups and retention, the three delivery guarantees (at most once, at least once, exactly once effect), and how to build a consumer that deduplicates and commits offsets safely.

Gap-fill lesson · ✦ 2026 · Advanced · ⏱ 120 min

By the end of this lesson you can

- Explain topics, partitions, offsets and consumer groups
- Choose a delivery guarantee and design the consumer to match
- Make a consumer idempotent with deduplication on event IDs
- Describe how Kafka fits with batch pipelines and stream processing

What Kafka is

Apache Kafka is a system for recording and distributing streams of events: a click, a payment, a sensor reading, an order status change. Producers write events to a **topic**; consumers read them. Kafka keeps the events for a set time, so consumers can read at their own pace, replay history, and join the stream later.

CASE STUDY **A payment that was counted twice**

A payments service publishes "payment captured" events. A consumer that updates the ledger crashed after writing to the ledger but before recording its progress. On restart it re-read the event and counted the payment twice. The fix was not to avoid crashes; it was to make the consumer's write idempotent, so that a repeated event changes nothing.

Topics, partitions and offsets

A **topic** is a named stream. Each topic is split into **partitions**: ordered, append-only logs. Events with the same key always go to the same partition, so their order is kept for that key. Each event in a partition has an **offset**, a sequence number that a consumer uses to say where it has read up to.

```
Topic: orders
  Partition 0:  [offset 0] [offset 1] [offset 2] ...   keys A, C, A
  Partition 1:  [offset 0] [offset 1] ...              keys B, B
```

Partitions give parallelism: more partitions let more consumers read at once. Order is guaranteed only within a partition, which is why the key matters.

Consumer groups

A **consumer group** is a set of consumers that share a topic's partitions, so each partition is read by one member of the group at a time. Adding consumers to the group spreads the load. A second, separate group reads the same topic independently, which is how one stream feeds several systems (the warehouse, a fraud check, a dashboard).

Retention

Kafka keeps events for a configured time or size, whether or not consumers have read them. Retention makes replay possible: a new consumer can start from the beginning, or a consumer can rewind after a bug fix. It is not a permanent archive; long-term storage belongs in the lake.

Delivery guarantees

Guarantee	Meaning	When it happens	Consequence
At most once	An event may be lost but is never processed twice	Offsets committed before processing	Loss on failure
At least once	An event is never lost but may be processed twice	Offsets committed after processing	Duplicates on failure
Exactly once (effect)	The result is as if each event were processed once	Idempotent processing, or transactions	Requires design or transactional features

Most systems run **at least once** and make the processing idempotent. That is simpler and more robust than trying to make delivery itself exactly once.

An idempotent consumer

The consumer below is written as a pattern, not a finished program. It records each event's ID in the same transaction as the business write, so a repeated event is recognised and skipped.

Python

```
# Illustrative pattern (not run: needs a Kafka client and a database).
# Shows the logic that makes at-least-once delivery safe.

def handle(event, db):
    with db.transaction():                          # one transaction for both writes
        if db.exists("processed_events", event["event_id"]):
            return "skipped (already processed)"    # a duplicate: do nothing
        db.execute(
            "INSERT INTO ledger (order_id, amount) VALUES (%s, %s)",
            (event["order_id"], event["amount"]),
        )
        db.execute("INSERT INTO processed_events (event_id) VALUES (%s)",
(event["event_id"],))
    return "processed"

# Offsets are committed only after handle() returns, so a crash causes a re-read,
# and the duplicate is skipped by the check above.
```

The key is that the business write and the record of processing succeed or fail together. If the record is written but the offset is not committed, the event is read again and skipped. If neither is written, the event is processed on the retry.

::::warn Commit offsets after the work, not before Committing the offset before processing turns a crash into a lost event. Commit after the work is safely stored, and make the work idempotent so that the repeat is harmless. ::::

Batch and streaming together

Kafka rarely replaces the warehouse. A common design lands the stream in the lake as files (for example, hourly Parquet partitions), and the batch pipelines of the earlier lessons process those files into reporting tables. Stream processing (Kafka Streams, Flink, Spark Structured Streaming) is used when the result must be updated within seconds, such as a fraud alert or a live stock count.

Need	Choose
Reports refreshed daily	Batch from files or tables
Reports refreshed hourly	Micro-batch from landed files
Alerts within seconds	Stream processing on Kafka
Feeding several systems from one source	Kafka topic with several consumer groups

Operational checks

- Consumer lag (how far behind the newest offset each group is) is the first health metric to watch.
- Track the number of duplicates skipped; a rising number signals retries or producer problems.
- Plan partition counts ahead, because increasing them later changes which partition each key goes to.
- Protect topics with access control; events often contain personal data.

INTERVIEW CORNER

How do you guarantee that a streaming consumer does not double-count?

Assume at-least-once delivery, because that is what most systems provide. Make each write idempotent by recording the event ID in the same transaction as the business write, skip events already recorded, and commit offsets only after the work is stored. Monitor consumer lag and the rate of skipped duplicates.

::::key What to remember - A topic is split into ordered partitions; order is guaranteed only within a partition, per key. - Consumer groups share the work and let several systems read the same topic independently. - Retention enables replay; it is not a permanent archive. - Design for at-least-once delivery and make processing idempotent. - Commit offsets after the work is stored, not before. ::::

Quick quiz

5 questions · tap an answer to check it

1. Where is event order guaranteed in Kafka?

A Across the whole topic

B Within a single partition, for events with the same key

C Between topics

D Nowhere

Answer: B. Partition-level order, with keys mapping events to partitions.

2. What does a consumer group do?

A Stores the data permanently

B Shares a topic's partitions among its members so each partition is read by one at a time

C Deletes old events

D Encrypts events

Answer: B. Consumer groups divide the work.

3. Which guarantee can lose events on failure?

A At most once

B At least once

C Exactly once (effect)

D Idempotent

Answer: A. Committing before processing risks losing work on a crash.

4. Why record the event ID in the same transaction as the business write?

A It makes the database faster

B So the write and the record of processing succeed or fail together, which prevents double counting

C Kafka requires it

D It removes the need for offsets

Answer: B. Atomic recording makes the duplicate check reliable.

5. Which metric is the first health check for a consumer group?

A The number of topics

B Consumer lag

C The message colour

D The size of the cluster only

Answer: B. Lag shows how far behind a group is.

EXERCISE

Design the offset policy

A consumer updates a customer's loyalty points from a Kafka topic. Describe the delivery guarantee you would design for, the idempotency mechanism, when offsets are committed, and what happens after a crash at each of three points: before the write, after the write but before the record, and after both.

Show solution (worked answer)

Expected: at-least-once delivery with idempotent writes; the event ID record in the same transaction; commit after success. Crash before the write: the event is re-read and processed. Crash after the write but before the record: impossible if they are in one transaction, and otherwise the duplicate is detected by the existence check. After both: the offset is committed late, the event is re-read and skipped.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.